University of Petra

307307
Generative AI

Introduction to Agentic AI

Introduction to Agentic AI

- Large Language Models (LLMs) are very good at producing text: answering questions, summarizing, drafting, and explaining. But on their own, they behave like a “single interaction” system: you ask once, they answer once.
- Many real tasks do not fit that shape. For example:
- “Find the latest information and cite sources.”
- “Analyze my dataset and compute statistics.”
- “Check this code for bugs, then propose a fix, then verify the fix.”
- These tasks require more than writing text. They require **doing steps**, **checking results**, and sometimes **using external systems** (like search engines, files, APIs, or code execution).
- **Agentic AI** is the approach of building AI that can handle this kind of work reliably: not only generating language, but also managing a multi-step process toward a goal.

The Problem With “One-Shot” LLM Use

- LLMs are powerful, but they have a structural limitation: they behave like a person who answers immediately, without doing any work outside the conversation. That is fine when the question is self-contained. It fails when the task requires *process*.
- A “one-shot” prompt struggles when you need any of the following:
- **Up-to-date facts** (the model may not know what happened yesterday).
- **Precise computation** (the model can make arithmetic or logic mistakes).
- **Verification** (the model cannot reliably prove it is correct).
- **Multi-step coordination** (the model may skip steps or lose track).
- **Interaction with systems** (files, APIs, databases, code, tickets, emails).
- This is why many people experience “confident but wrong” answers. The model is optimizing for plausible text, not for completing a real workflow with checks, evidence, and state.
- Agentic AI exists to solve exactly this gap: it adds structure around the model so tasks can be executed as a process rather than a single response.

What We Mean by “An Agent” (Plain Definition)

- An **agent** is an AI system that is given a goal and can make progress toward that goal by deciding what to do next.
- In simple terms, an agent is like an assistant that can:
  - read your instruction,
  - decide the next step,
  - and continue step-by-step until it reaches a usable result.
- The difference from a normal chatbot is not “intelligence.” The difference is **behavior over time**:
  - A chatbot gives an answer and stops.
  - An agent keeps working until the goal is achieved, or it decides it cannot proceed.
- This does not mean the agent is “independent” or “uncontrolled.” In well-designed systems, the agent operates inside boundaries: limited permissions, limited tools, and clear stopping conditions.

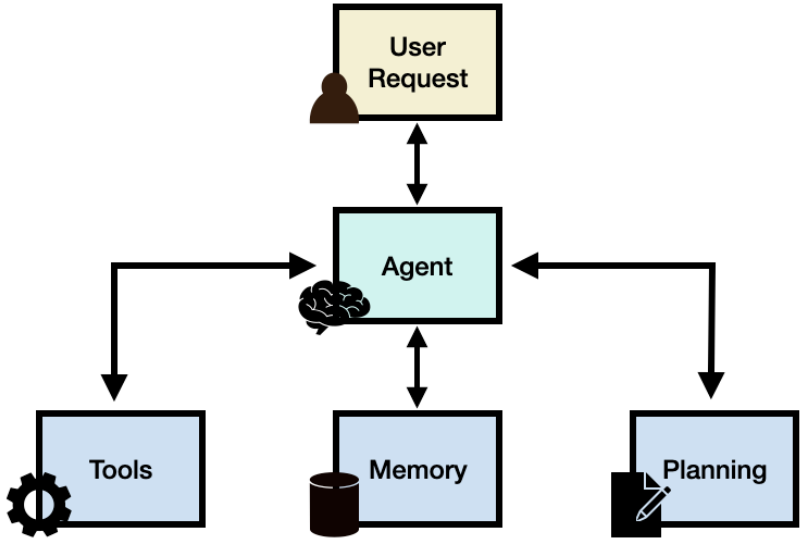
The Building Blocks of Agentic AI

Agentic AI is not a single model feature. It is a system built from a few common building blocks:

1) The Language Model (the brain)
This is the component that understands instructions, generates plans, and makes decisions in natural language.

2) Tools (the hands)
Tools are external capabilities the agent can use to do real work, such as:

- searching the web,
- calling an API,
- reading a file,
- running Python code,
- querying a database.



3) State / Working Context (the scratchpad)
To do multi-step work, the agent needs a place to keep track of:

- what it already tried,
- what it learned,
- intermediate results,
- what still needs to be done.

4) Control Rules (the boundaries)
Real agents must follow rules such as:

- what tools are allowed,
- when to stop,
- when to ask for clarification,
- what counts as “done.”

- If students remember only one thing from this slide: agentic AI works because the model is combined with tools, state, and control—not because the model magically became autonomous.

A Simple Example: “Find, Compute, Explain”

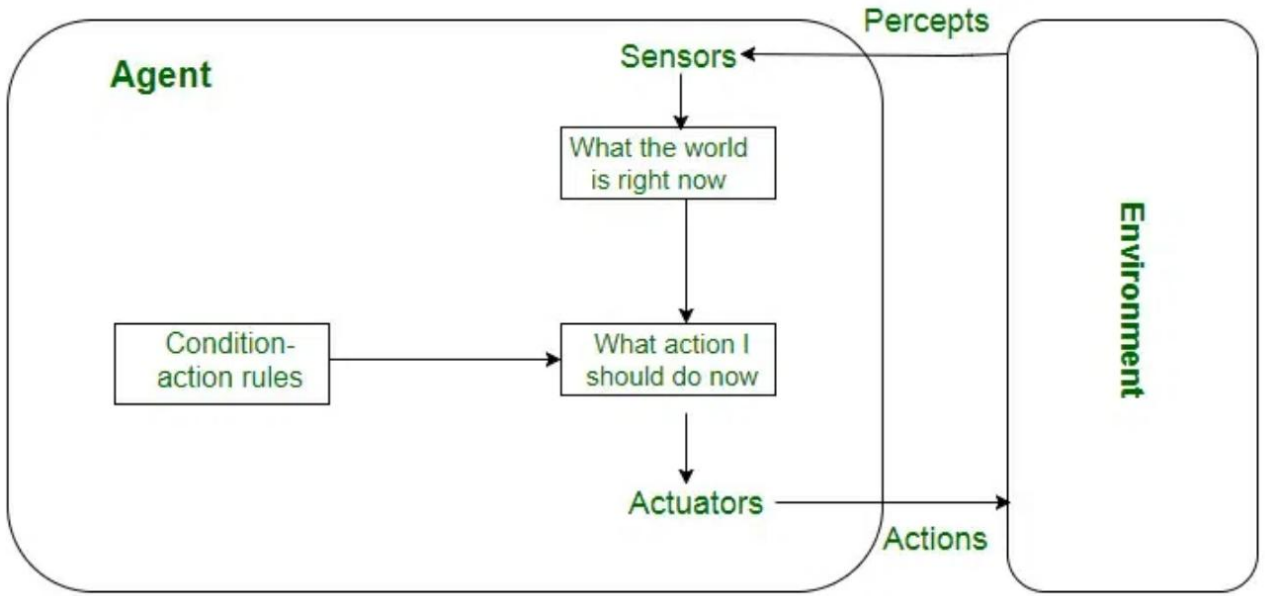
To make the idea concrete, consider a task that looks easy but actually has multiple parts:

- **Task:** “Find the population of Amman, compute what 3% of it is, and explain what that number means.”
- A normal LLM might:
  - guess a population number,
  - do a rough calculation,
  - produce a confident explanation.
- An agentic system would treat this as a process:
  - **Find the population from a reliable source** (search tool).
  - **Compute 3% precisely** (calculator or Python tool).
  - **Write the explanation** (LLM).
  - **Include the source and the computed result**.
- The key idea: the writing part is only the final step. The correctness comes from using the right tool at the right time, and keeping track of results.

The “Agent Workflow”

When an agent solves a task, it usually follows a repeated pattern:

- It reads the goal.
- It decides (PLANS): “What is the next step that moves me forward?”
- If needed, it uses a tool to get information or perform an action.
- It looks at the result and decides what to do next.



This repeats until one of these outcomes happens:

- The goal is achieved (a usable final result).
- The agent needs clarification (missing information).
- The agent is blocked (no permission, no tool, or unreliable data).

This is the practical meaning of “agents work step-by-step.” It is not magic. It is structured progress.

Code: Setup + Agent Decision

```
import pandas as pd
import google.generativeai as genai
import json

# Load dataset
df = pd.read_csv("sales.csv")   # Example dataset

# Configure Gemini
genai.configure(api_key="YOUR_API_KEY")
model = genai.GenerativeModel("gemini-1.5-flash")

# Tools the agent can use
def summary_tool(_):
    return df.describe().to_string()

def correlation_tool(_):
    return df.corr(numeric_only=True).to_string()

tools = {
    "summary": summary_tool,
    "correlation": correlation_tool
}
```

Code: Setup + Agent Decision

```
# Agent decides next action
def agent_decide(state):
    prompt = f"""
    You are a data analyst agent. Based on the task and dataset preview below,
    choose the best next analytical action.

    Return JSON only:
    {{"action": "..."}}

    State:
    {state} """
    return model.generate_content(prompt).text
```

Code: Execute Tool + Final Interpretation

```
def execute(action_json):
    data = json.loads(action_json)
    action = data["action"]


    if action in tools:
        return tools[action](None)
    else:
        return "Unknown action"


# Initial state includes task + preview of data
state = f"""
Goal: Provide insights about the sales dataset.
Data preview:
{df.head().to_string()}
"""
```

Code: Execute Tool + Final Interpretation

```
# Step 1: Agent chooses action
decision = agent_decide(state)
print("Model decision:", decision)

# Step 2: Python executes tool
observation = execute(decision)

# Update state
state += f"\nExecuted: {decision}\nObservation:\n{observation}"

# Step 3: Agent provides final insights
final_prompt = f"""
You are a senior data analyst.
Based on the updated state, produce a short, clear explanation of the insights.
"""

final_answer = model.generate_content(final_prompt).text

print("\nFinal Insight Report:\n", final_answer)
```